

CPSC 375 High-Performance Computing

Lecture 5

Shared Memory

Linux IPC

- Pipes
- Message Queues
- **Shared memory**
- Semaphores
- Signals
- Sockets

What Is Shared Memory?

- Specific region of physical RAM mapped into the virtual address spaces of multiple processes
- Multiple processes granted access to the same region
 - Memory vs. messaging (pipes, message queues)
- Independent of the processes that use shared memory
- Provides raw speed but lacks synchronization
 - Programmer's responsibility

Performance

- Pipes and message queues require copying through kernel buffers
- Shared memory operates in user space
 - Can achieve the RAM bus limit: ~50 GB/s
 - Critical for HPC applications moving terabytes of data.
- Avoids system call overhead: `read()` and `write()`
 - CPU must trap to the kernel, save registers, etc.
- Reduces the number of context switching
 - Can communicate with one another w/o ever notifying kernel.
 - Reduces the total CPU cycles spent on admin tasks vs actual computation

Data Movement

- Typical IPC data flow

Process A → Kernel Buffer → Process B

- CPU spends cycles moving bytes rather than computing
 - Moving a 1GB matrix, the CPU must execute instructions to copy that 1GB twice
- Tripling requirements
 - Data in Process A, kernel buffer, and Process B simultaneously
 - Large-scale data processing hampered by high latency and excessive memory consumption
- Shared memory eliminates the kernel from the data path
 - Kernel's only role is to set up the map

Zero-Copy

- Processes access the same physical memory frames directly
 - Data never moves from its original physical location.
 - OS essentially creates a "shortcut" in the virtual memory of both processes
- Data access happens instantly because no data actually moves
 - Perfect for massive arrays, or matrices for scientific computing.
 - For Big Data applications where copying is physically impossible
- Must guarantee *data serialization*
 - Must ensure that one process doesn't read the data while another process is halfway through writing it

Shared Memory Use Cases

- Database
 - Multiple backends accessing a shared cache or buffer pool (*transaction processing*)
- Scientific simulations
 - Sharing a massive grid of coordinates between “neighboring” processes
- Task pools in HPC systems
 - shared array as a high-speed task list for a pool of processes

Virtual Memory (Review)

- Processes see virtual addresses, not physical RAM locations.
- *MMU* translates these addresses on the fly.
- Page tables track virtual *pages* to physical *frames*.
- By default, processes cannot see or access the memory space of other processes.

Memory Mapping

- OS maps the same physical frame to two different page tables.
 - To enable sharing, the kernel adds a pointer to the same physical frame in page tables of both processes.
 - e.g., Process A sees the buffer at 0x7000, while Process B sees it at 0x4000.
 - Both pointers lead to the exact same location on the RAM.
- Kernel establishes the mapping and stays out of the way during execution.

Memory Protection

- `PROT_READ`
 - Allows the process to read from the memory segment.
- `PROT_WRITE`
 - Allows the process to modify the segment.
- Generates a segmentation fault if a process violates.
- Best practice: read-only segments for consumer processes

Where Is the Shared Memory?

- Temporary filesystem
 - Shared objects are visible in `/dev/shm` on Linux.
 - Allows you to use standard commands like `rm`, `ls`, and `chmod`
 - e.g. to check size and ownership:

```
$ ls -l /dev/shm
```
- Volatile storage: (`tmpfs`): disappears if the system reboots.
- OS restricts the total amount of RAM for shared memory.
 - Usually 50% of RAM
 - Important to unlink when finished with shared memory, or the system will run out of space on device.

Allocation vs. Mapping

- Allocation
 - Kernel reserves physical RAM with a name
 - New segment dedicated to shared object
 - can exist without any process being currently attached
- Mapping
 - Process connects its virtual address space to that segment.
 - OS modifies page tables to reflect the request.
- Master vs. worker processes
 - Master creates and populates memory
 - Worker attaches to it anytime during its lifespan
- Kernel keeps the segment alive as long as it is linked or mapped.
 - Tracks the number of processes currently mapped on the segment.
 - Won't automatically delete the segment when processes exit unless `shm_unlink()` was called.

Create/Open Shared Object

```
int shm_open(char *name, int flag, mode_t mode);
```

name: string beginning with /

flag: O_CREAT (create), O_RDWR (read/write), or O_EXCL.

mode: permissions (e.g., 0666) used only if creating the object.

- Returns a file descriptor (handle) on success, or -1 on error.
- This initiates the connection to the kernel's IPC system.
- It doesn't allocate memory yet.

Set the Size of Shared Object

```
int ftruncate(int fd, off_t length);
```

`fd`: **File descriptor** returned by `shm_open`.

`length`: **Size of the memory segment in bytes.**

- New objects have a size of 0 by default.
- Without this call, any attempt to map or write to it will result in a bus error or segmentation fault.

Map Shared Object

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

addr: **usually NULL (let the OS choose)**

length: **number of bytes to map.**

prot: **protection bits (PROT_READ | PROT_WRITE)**

flags: **must include MAP_SHARED for IPC**

fd: **handle from shm_open**

offset: **starting point in the file (usually 0)**

- **Returns a pointer to the shared memory to be used to read and write directly to the shared memory.**

Unmap Shared Memory

```
int munmap(void *addr, size_t length);
```

addr: **pointer returned by** `mmap`

length: **size of the mapped region**

- Unmaps (detaches) the memory from the calling process's address space
- This process can no longer access the shared memory
- Data survives in the kernel for other processes to use

Remove Shared Object

```
int shm_unlink(const char *name);
```

name: **string name used in shm_open**

- **Removes the name from /dev/shm**
- **The memory is physically reclaimed by the kernel only after the last process currently using it calls `munmap` or terminates.**

Don't Forget to `ftruncate()` !

- New shared memory objects have a size of 0 bytes by default.
- Attempting to `mmap` a 0-byte segment results in a bus error.
- Once set, the size is fixed until unlinked.
 - Technically call `ftruncate` again to resize, but in most HPC applications, the size is set once at the start of the simulation.

Example: The “Master” Process

```
// Create the handle to the segment
int fd = shm_open("/mysm", O_CREAT | O_RDWR, 0666);

// Set the size
ftruncate(fd, 4096); // page size on Linux systems

// Map to the segment
char *pch = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

// Write data to the segment
sprintf(pch, "Hello, World!");

// Cleanup
munmap(ptr, 4096);
shm_unlink("/mysm");
```

Example: The “Worker” Process

```
// Open handle to an existing segment
int fd = shm_open("/mysm", O_RDONLY, 0666);

// Map to local pointer
char *pch = mmap(0, 4096, PROT_READ, MAP_SHARED, fd, 0);

// Read from the segment
printf("From SM: %s\n", pch);

// Cleanup
munmap(pch, 4096);
```

Named vs. Unnamed Shared Memory

- Accessible via a string name like `/my_sm`.
 - Used for unrelated processes. It's easy to use and visible in `/dev/shm`.
- Unnamed shared memory: Created via `MAP_ANONYMOUS`.
 - This has no name in the kernel.
 - It is only accessible to a parent process and its children created via `fork()`.
 - e.g., parallelized search where one process forks into four workers
- Anonymous shared memory vanishes automatically when the last child exits
 - Nothing to unlink

Need for Synchronization

- Kernel provides no protection
 - OS assumes you know what you are doing (?)
- What if two processes write to the same byte simultaneously?
 - Interleaved bits and corrupted data
 - Unlike a message queue, where messages stay whole, shared memory can result in partial writes
- Speed comes with the price of complexity.
 - Cannot use shared memory effectively without a synchronization plan

Race Condition

- Three-step CPU cycle for `counter++`: Read-Modify-Write
 1. Load value from memory to a register.
 2. Increment the register.
 3. Store the register back to memory.
- OS context switching between steps
 - If Process A is at step 2, and the OS switches to Process B, and
 - Process B finishes all 3 steps.
 - When Process A resumes, it will overwrite Process B's work.
- Two processes both incremented the counter, but the final value only went up by one.
- Solution: Next time

